

Rapport de benchmark

BenchmarkTick-16 — analyse CPU, mémoire et hyperfine

Généré le	2026-09-24T08:26:18.2250847Z
Commande	go test ./game -run '^\$' -bench '^BenchmarkTick\$' -benchmem -count 1
Plateforme	windows/amd64
Processeur	AMD Ryzen 7 7735HS with Radeon Graphics
Profil CPU	cpu-20260924-082618232.prof
Profil mémoire	memory-20260924-082618232.prof

1. Résultats du benchmark

Benchmark	Moyenne	ns/op	B/op	allocs/op
BenchmarkTick-16	122.86 ms	122861356	0	0

Détail hyperfine

Métrique	Valeur
Moyenne	3.3699 s
Écart-type	n/a
Médiane	3.3699 s
Temps utilisateur	2.7303 s
Temps système	0.6516 s
Min	3.3699 s
Max	3.3699 s

Résumé CPU et mémoire

Mesure	Valeur
CPU échantillonné (pprof)	1.16s
Mémoire allouée (pprof)	19308.28kB
Mémoire par opération	0 B/op

Garbage collector Go

Cycles GC	57
Temps GC cumulé	49.940 ms

Plus longue pause	2.700 ms
Temps CPU GC cumulé	272.390 ms
Objectif mémoire maximal	97.00 MB

2. Profil CPU — où passe le temps ?

Type de profil : **cpu**. Total échantillonné : **1.16s**.

Fonction	Flat	Flat %	Cumulé	Cumulé %
jeu-de-la-vie/game.(*Board).shouldInfect	1s	86.21%	1.12s	96.55%
jeu-de-la-vie/game.(*Board).index (inline)	0.06s	5.17%	0.06s	5.17%
jeu-de-la-vie/game.squaredDistance (inline)	0.05s	4.31%	0.05s	4.31%
runtime.cgocall	0.02s	1.72%	0.02s	1.72%
jeu-de-la-vie/game.(*Board).Step	0.01s	0.86%	1.14s	98.28%
math/rand.(*Rand).Int63 (inline)	0.01s	0.86%	0.01s	0.86%
runtime.memmove	0.01s	0.86%	0.01s	0.86%
internal/syscall/windows/registry.Key.GetMUISt	0	0.00%	0.02s	1.72%

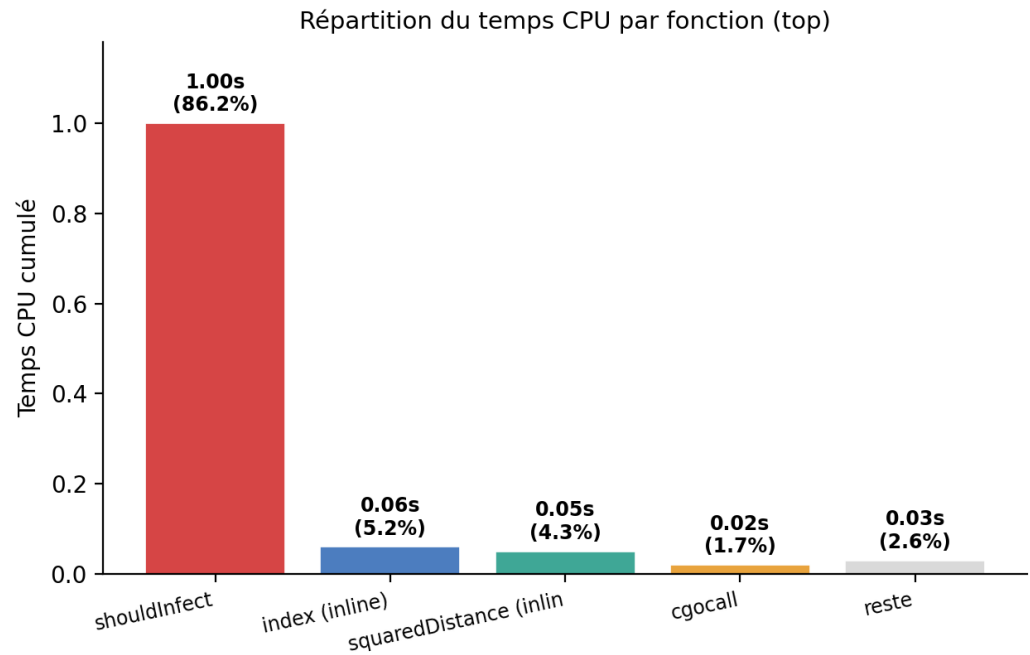


Figure — Répartition du temps CPU par fonction (temps flat, top fonctions).

File: game.test.exe
 Build ID: C:\Users\saiou\JeuDeLaVie\game.test.exe2026-09-24 10:23:54.3290161 +0200 CEST
 Type: cpu
 Time: 2026-09-24 10:26:18 CEST
 Duration: 1.17s, Total samples = 1.16s (99.05%)
 Showing nodes accounting for 1.16s, 100% of 1.16s total
 Showing top 15 nodes out of 33
 See <https://git.io/HYMW> for how to read the graph

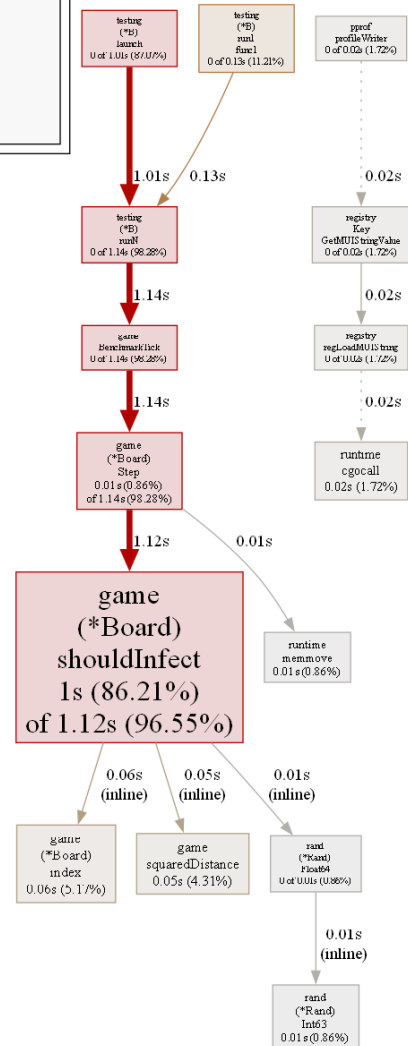


Figure — Graphe d'appels CPU (go tool pprof -png).

2.1 Diagnostic automatique

La fonction dominante de ce run est **jeu-de-la-vie/game.(*Board).shouldInfect**, responsable de **86.2%** du temps CPU total (flat), pour **96.5%** cumulé avec ses appels. C'est la cible prioritaire d'optimisation pour ce run.

- Le nom de cette fonction suggère une recherche/un parcours répété (distance, voisinage, scan...) — vérifier si elle itère sur l'ensemble des données à chaque appel plutôt que sur un sous-ensemble borné (structure spatiale : grille, quadtree, index).
- Comparer ce hotspot avec le run précédent : s'il a changé de fonction, l'optimisation précédente a fonctionné et un nouveau goulot est apparu ailleurs.
- Vérifier s'il existe un algorithme en $O(n)$ ou $O(\log n)$ équivalent pour cette opération avant d'optimiser au niveau micro (inlining, allocations).

Rappel — pourquoi la complexité algorithmique domine
à mesure que les données grandissent (schéma, non mesuré)

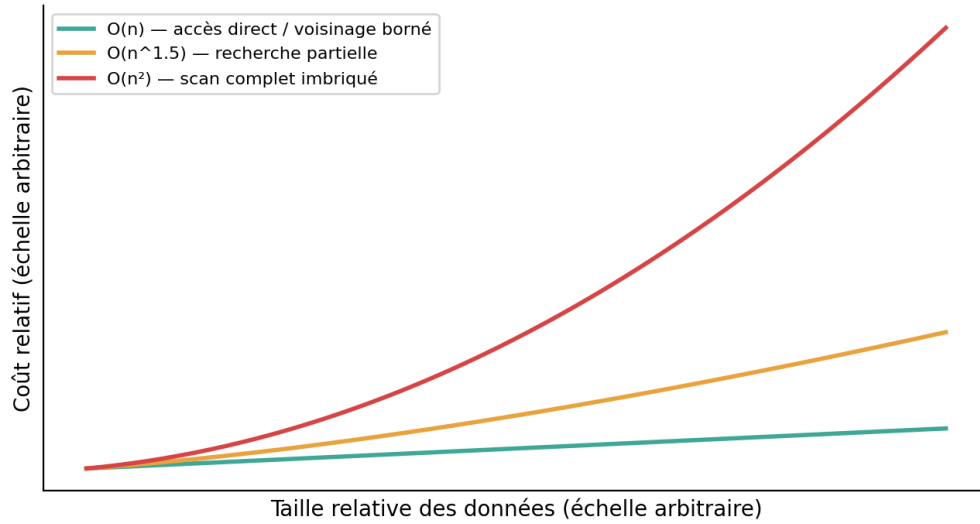


Figure — Rappel pédagogique sur la complexité algorithmique (schéma générique).

3. Profil mémoire — allocations

Type de profil : `alloc_space`. Total : **19308.28kB**.

Fonction	Flat	Flat %	Cumulé	Cumulé %
jeu-de-la-vie/game.NewBoard (inline)	8487.14kB	43.96%	8487.14kB	43.96%
jeu-de-la-vie/game.RandomBoard	5655.11kB	29.29%	14142.25kB	73.24%
runtime/pprof.StartCPUProfile	1184.27kB	6.13%	1184.27kB	6.13%
math/rand.newSource (inline)	1029.26kB	5.33%	1029.26kB	5.33%
compress/flate.NewWriter (inline)	902.59kB	4.67%	902.59kB	4.67%
runtime.mallogcgc	513kB	2.66%	513kB	2.66%
runtime/pprof.(*profileBuilder).emitLocation	512.88kB	2.66%	512.88kB	2.66%
syscall.UTF16ToString	512.02kB	2.65%	512.02kB	2.65%

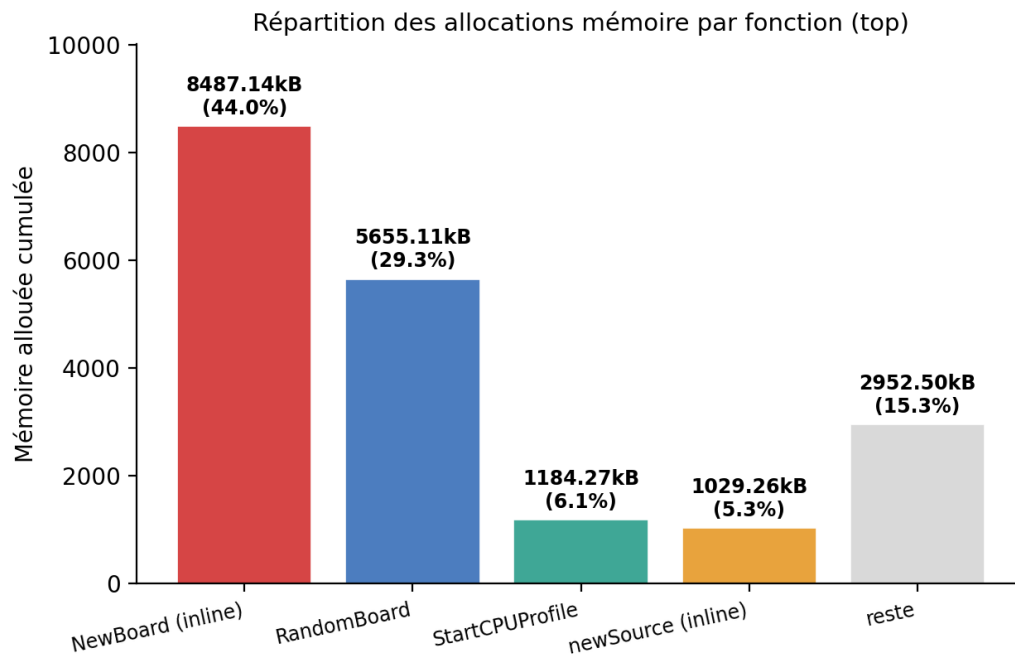


Figure — Répartition des allocations mémoire par fonction.

See <https://git.io/JfYMW> for how to read the graph

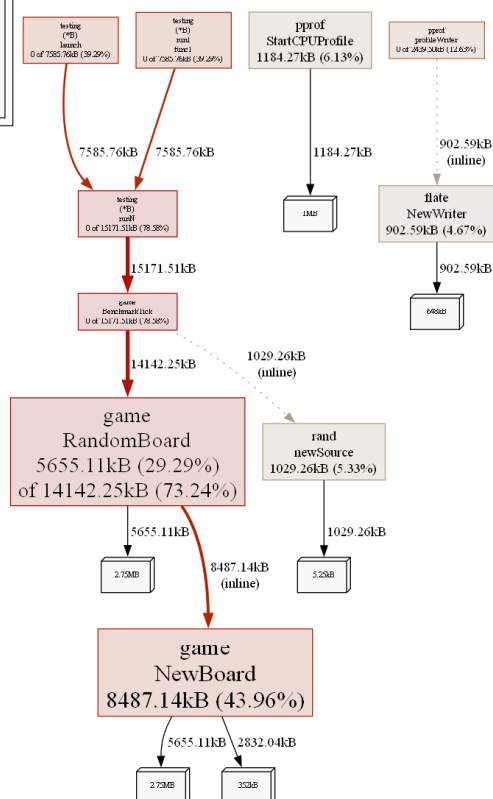


Figure — Graphe d'appels mémoire (go tool pprof -png).

4. Limites de ce benchmark & informations manquantes

Générées automatiquement à partir de la configuration détectée :

- **Peu de runs (1).** Avec moins de 5 exécutions, aucune médiane/écart-type fiable n'est disponible. Recommandé : ≥ 10 runs (-count=10 ou hyperfine --min-runs 10).
- **Aucun warmup.** La première exécution peut être biaisée (cache froid, JIT du GC non stabilisé). Ajouter 1-2 warmups avant la mesure.
- **Pas de comparaison avant/après** intégrée à ce rapport — conservez les PDF successifs pour comparer les hotspots et le temps moyen dans le temps.
- **Un seul point de taille de données.** Si applicable, mesurer plusieurs tailles permettrait de vérifier empiriquement la complexité algorithmique observée en section 2.

Notes de l'outil de benchmark

- Mesure du calcul du tick sur une carte 600×600 .
- La carte et le générateur aléatoire utilisent toujours la seed 42.
- La latence API est mesurée à part dans le frontend, sans rendu canvas.
- La trace GC détaillée est enregistrée dans le fichier `gc-\*.log` et résumée dans le PDF.
- Les métriques I/O, réseau et base de données ne sont pas instrumentées dans ce benchmark.

Rapport généré automatiquement le 2026-09-24 10:26:27 par generate_benchmark_report.py, à partir des fichiers les plus récents du dossier de benchmarks. Analyse réalisée avec go tool pprof.